USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 0 Verification Report

UM-NATOS-006

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-006	1.0 · 2026-08-14	**PASS** — all assertions verified on hardware	Internal

1. Abstract

Milestone 0 establishes that nat-os owns the machine: that a from-scratch image is loaded by the bootloader, that its segments arrive at the intended addresses, that the entry stub produces a usable C environment, and that the kernel keeps running. This report records the test method, the raw captured output, and a pass/fail determination for each assertion.

Every milestone after this one assumes these five facts. If a later milestone fails inexplicably, re-running this test is the correct first step.

2. Why M0 asserts rather than prints "hello"

A conventional first program prints a greeting. That proves the CPU reached the kernel, and nothing else. It does not distinguish a working image from one where `.data` was never copied, `.bss` holds reset garbage, or code is executing from an unintended region.

Those three failures are the characteristic failure modes of a hand-written linker script, and each produces symptoms that surface much later, in unrelated code, as inexplicable corruption. M0 therefore checks them explicitly at boot and reports each result, so a broken link map is caught at the point of change rather than during scheduler debugging.

This matters more than usual here because **no JTAG probe is yet available** (UM-NATOS-007 §7). Self-reporting is the only diagnostic channel.

3. Test configuration

Item	Value
Date	2026-08-14
Target	ESP32 CYD board, fresh unit, COM5
Image	`build/natos.bin`, 1,216 bytes
Bootloader	Borrowed, 17,536 bytes @ `0x1000`
Partition table	Borrowed, 3,072 bytes @ `0x8000`
Toolchain	`xtensa-esp32-elf-gcc` 14.2.0, `-mabi=call0`
Capture	Port opened before reset; auto-reset pulsed; 8 s window

Flash write verified by `esptool`: all three regions reported "Hash of data verified."

4. Raw captured output

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4


====================================
 nat-os  milestone 0 — kernel alive
====================================
  .data loaded : ok (0xc0deface)
  .bss cleared : ok
  bss span     : 0x3ffb0188 .. 0x3ffb018c (4 bytes)
  stack top    : 0x3ffdc200
  code at      : 0x40080088  (IRAM ok)

  no scheduler yet — next: timer interrupt + context switch
  heartbeat: 0 1 2 3 4 5 6
```

694 bytes received over the 8-second window.

5. Assertions

#	Assertion	Method	Result
1	Image header is well-formed and the bootloader accepts it	Banner appears at all; `esptool` reported checksum and SHA-256 valid	**PASS**
2	`.data` segment was copied to DRAM	Canary initialised to `0xC0DEFACE` in `.data`, read back at runtime	**PASS** — read back `0xc0deface`
3	`.bss` was zeroed by `start.S`	Canary in `.bss` compared against zero	**PASS**
4	Code executes from the declared IRAM window	Address of a function compared against `0x40080000` – `0x400A0000`	**PASS** — `0x40080088`
5	Stack is usable; call/return works under call0	Every self-check is a C function call that returned	**PASS** (implicit)
6	Kernel remains running	Heartbeat counter advanced 0→6 over the capture window	**PASS**

5.1 Supporting observations

- `rst:0x1 (POWERON_RESET)` — a clean cold boot. Not a watchdog (`rst:0x3` / `0xc`) or panic reset, so the kernel did not crash and restart.
- **`.bss` span** `0x3FFB0188` – `0x3FFB018C` — 4 bytes, matching the single declared `.bss` canary, and located in the DRAM region declared by the linker script.
- **Stack top** `0x3FFDC200` — equals `ORIGIN(dram) + LENGTH(dram)` = `0x3FFB0000 + 0x2C200`, confirming the linker script's arithmetic reached the running image.

6. What this milestone does not establish

Stated explicitly so later work does not over-claim on this evidence.

- **No interrupt handling.** No vector table is installed. Behaviour on any exception or interrupt is undefined.
- **No timing accuracy.** The heartbeat uses a spin loop of arbitrary length against an unknown CPU clock. The counter proves liveness, not rate.
- **No watchdog management.** The kernel neither feeds nor disables any watchdog. That it survived 8 seconds suggests watchdogs are not armed at this point in boot, but this is inference, not measurement, and may change once interrupts are enabled.

> **CORRECTED 2026-08-14.** The inference was wrong. The RTC watchdog **is** armed by the second-stage bootloader, which expects the application to take ownership of it. M0 survived its capture window by luck of timing, not because nothing was running. Measured directly during M2: `rst:0x10 (RTCWDT_RTC_RESET)` on every boot once the CPU was kept busy. See UM-NATOS-009 §8 and `kernel/watchdog.c`. - **No memory beyond the first few bytes.** DRAM and IRAM were exercised only at their lowest addresses. Neither region has been validated across its full declared length — directly relevant to the overlap risk in UM-NATOS-004 §5. - **Single-core only.** Core 1 is untouched and in an unknown state.

7. Defects found during bring-up

Defect	Stage	Resolution
PowerShell parsed the comma in `-Wl,--gc-sections` as an array separator	Link	Quote all `-Wl,` arguments
`elf2image` failed with `No module named 'serial'`	Package	Use PlatformIO's `penv` Python; `esptool` imports `serial` even offline

Neither defect involved kernel source. Both are recorded in UM-NATOS-005 §9.

8. Conclusion

Milestone 0 passes. nat-os boots from flash on target hardware, its segments load where the linker script directs, the entry stub produces a working C environment under the call0 ABI, and execution is sustained.

The kernel is 1,124 bytes of text with no ESP-IDF, Arduino, FreeRTOS, or C library linked. Every instruction from `0x4008000C` onward is project code.

Blocking item before M1 is flashed: the bootloader IRAM overlap documented in UM-NATOS-004 §5. The kernel's `.text` is close to the threshold at which segment loading would overwrite the executing bootloader, and M1 will cross it.

9. References

- UM-NATOS-002 — boot chain and interpretation of the ROM trace
- UM-NATOS-004 §5 — the open overlap risk
- UM-NATOS-005 §8 — capture harness